

*Analyse et Conception des Systèmes d'Information*  
*Modèle logique de données Relationnel.*

*Michel EBOUEYA*  
*Université de La Rochelle*

# Sommaire

---

- Niveau logique
  - Fichiers
  - Bases de Données Relationnelles
  - Convention de traduction vers le niveau logique a travers un AGL
- Formes Normales

# Niveau Logique : pourquoi?

---

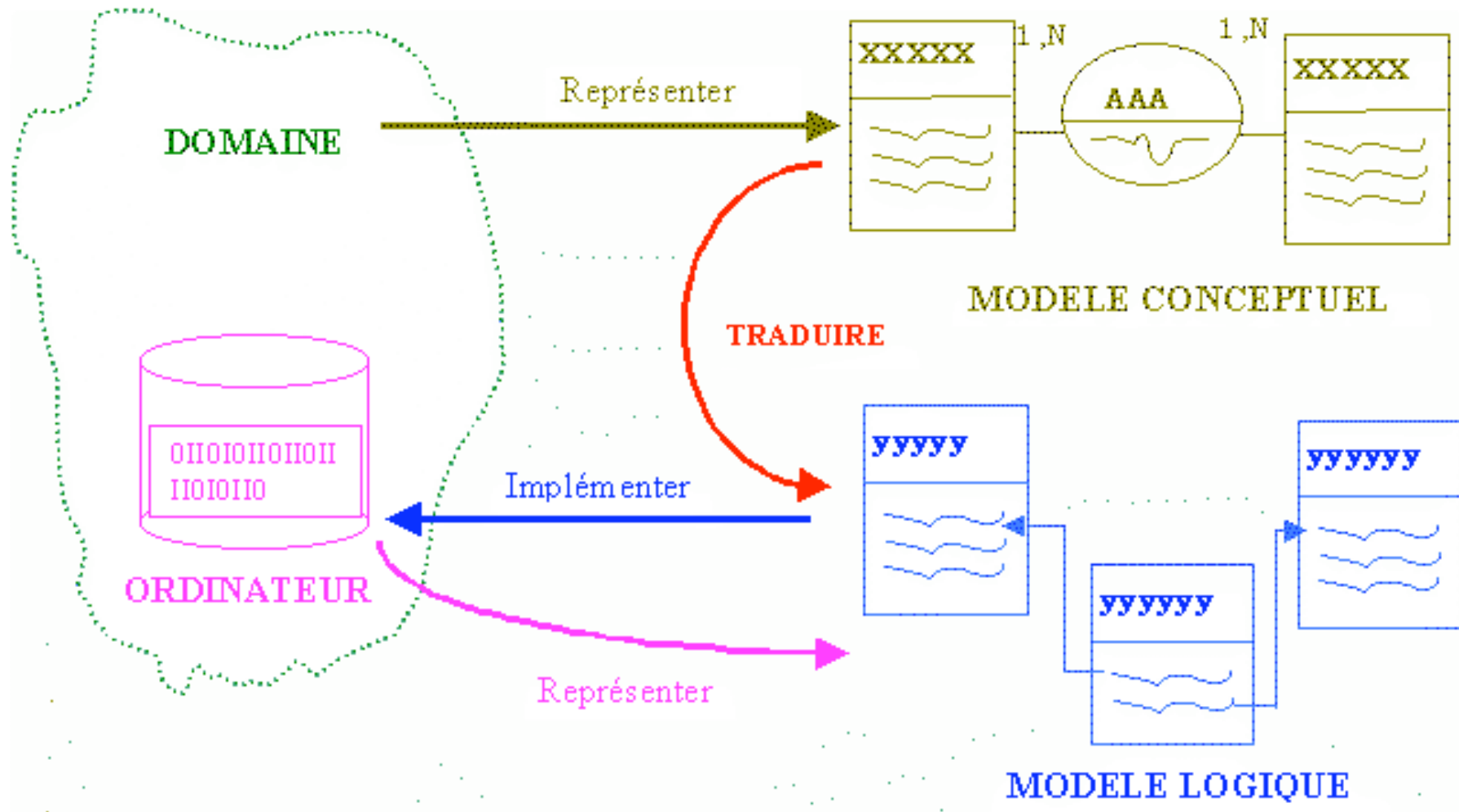
- On ne sait pas implémenter aisément un modèle Entité-Association dans une machine :
  - une association de dimension supérieure à 2 doit être transformée
  - une association porteuse de données n'est pas toujours implémentable telle quelle.
- Un modèle "conceptuel" de données est indépendant de l'état de l'art technologique.
  - Or, il existe différentes sortes de Systèmes de Gestion de Bases de Données (SGBD) qui ont chacun leur propre modèle (dans l'ordre historique) :
    - Systèmes de Gestion de Fichiers ou SGF qui ne sont pas vraiment des SGBD
    - SGBD hiérarchiques organisés selon une arborescence
    - SGBD réseau encore appelés CODASYL (du nom de la norme à laquelle ils se réfèrent)
    - SGBD relationnels
    - SGBDOO (Orientés Objet)

# Niveau Logique ?

---

- QU'EST-CE QUE LE NIVEAU LOGIQUE ?
- Le niveau logique est une REPRESENTATION du système tel qu'il sera implémenté dans des ordinateurs.
- Vu qu'il s'agit d'une représentation, on utilisera également un modèle et :
  - il ne faut pas confondre le modèle conceptuel (entité - association par ex.)
  - avec le modèle logique (relationnel par exemple)
  - il ne faut pas confondre le modèle logique (relationnel par ex. )
  - avec son implémentation physique en machine ( Access, Oracle par ex.)

# modèle conceptuel ---traduction ---->au modèle logique



---

Le modèle logique, en tant qu'il représente aussi un système informatique doit faire une hypothèse minimale sur l'état de l'art technologique qu'il prend en considération.

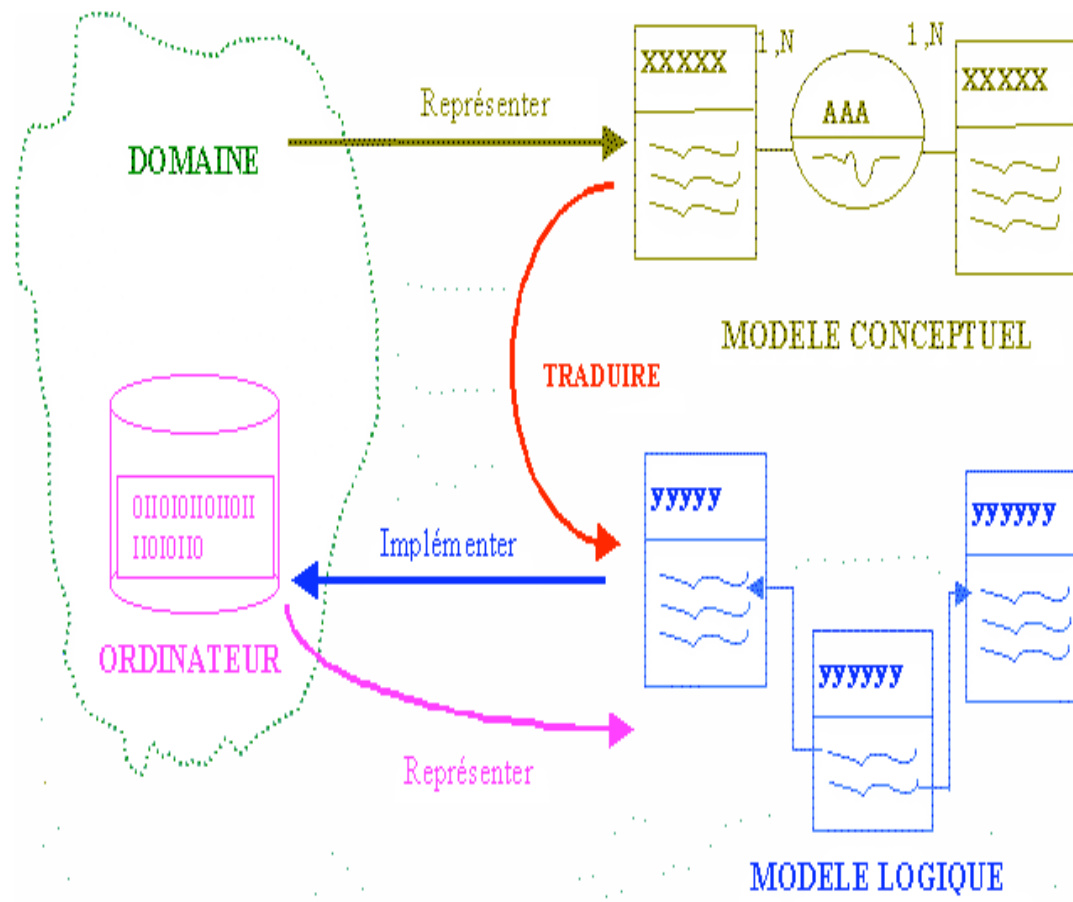
On aura donc un modèle relationnel différent d'un modèle CODASYL ou encore différent d'un modèle OO.

Mais s'il s'agit d'un modèle relationnel par exemple, il devra être indépendant d'une "plate-forme" particulière (Oracle, Postgres, etc...).

Bien qu'il doit déjà compliqué, ce schéma est incomplet.

Il représente la vue de la CONCEPTION du système et pas celle de son UTILISATION : le NIVEAU EXTERNE que vous appellerez VUES EXTERNES en cours de BD.

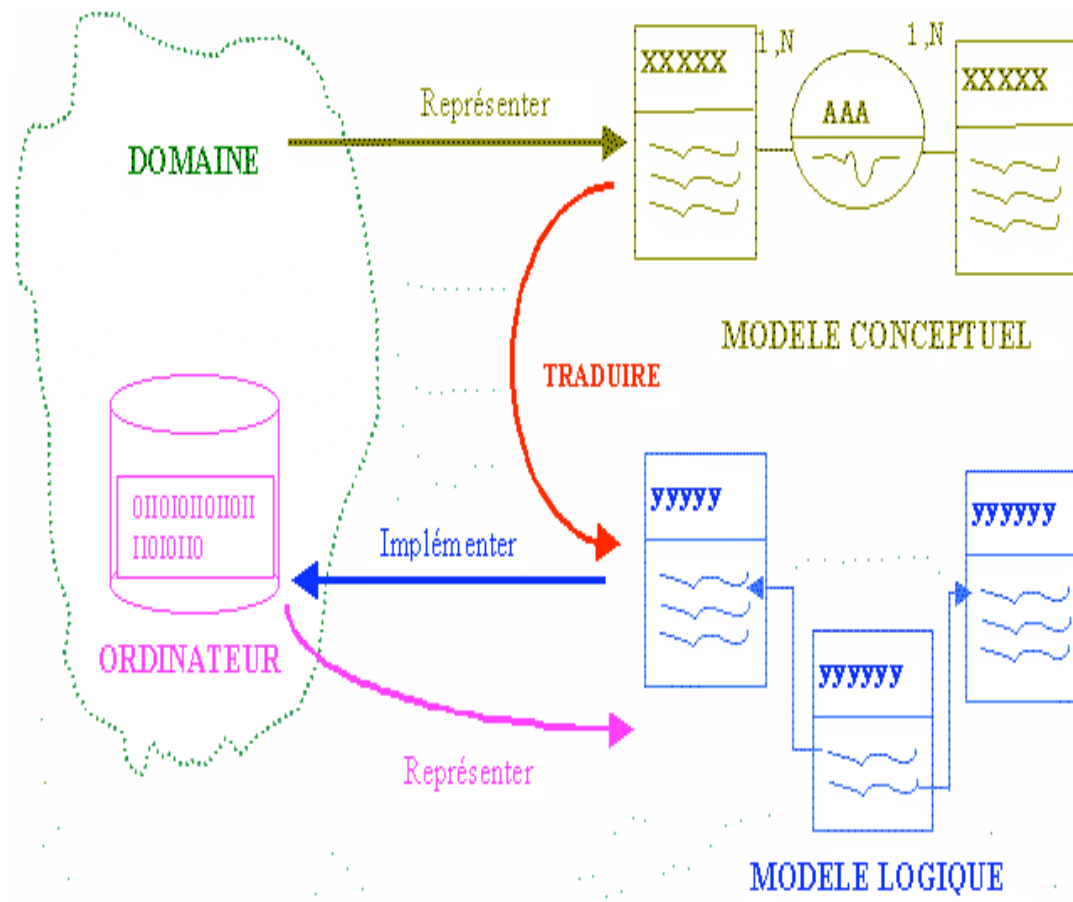
(Ce niveau suppose que l'on ait étudié l'activité du système et pas seulement ses données).



Bien qu'il doit déjà compliqué, ce schéma est incomplet.

Il représente la vue de la CONCEPTION du système et pas celle de son UTILISATION : le NIVEAU EXTERNE que vous appellerez VUES EXTERNES en cours de BD.

(Ce niveau suppose que l'on ait étudié l'activité du système et pas seulement ses données).



---

Nous ne donnons ici que les quelques notions élémentaires indispensables à la suite de ce cours :

Elles seront développées dans le cours BASES DE DONNEES.

DEFINITION EN EXTENSION D'UNE RELATION :

Une RELATION est une partie finie du produit cartésien de  $n$  ensembles (ou domaines)

Rien à voir, donc, avec la définition d'une ASSOCIATION.

# Définition en extension d'une relation

- Une RELATION est une partie finie du produit cartésien de n ensembles (ou domaines)
- Rien à voir, donc, avec la définition d'une ASSOCIATION.
- Un attribut prend ses valeurs dans un DOMAINE (soit l'ensemble de ses valeurs possibles)
- Ainsi, une RELATION (ou TABLE) est le résultat d'une opération sur les ensembles
- constitués par les valeurs des attributs.

| CODE_CLIENT | NOM    | AD_RUE            | AD_VILLE  |
|-------------|--------|-------------------|-----------|
| AXV508      | DUPONT | 15, rue du Canada | CAEN      |
| BWY405      | DURAND | 6, rue Méchin     | CHERBOURG |
| KAC567      | DUVAL  | 25 rue Blanche    | ALENCON   |

**ATTRIBUT**

**Valeurs d'attribut**

**N-UPLET**

# Définition en extension d'une relation

- CLE de relation :
  - un attribut particulier tel qu'il ne peut exister qu'une seule valeur de cet attribut pour tous les n-uplets de la table. Ce pourrait être ici le cas de l'attribut Code\_client. Cf identifiant.
- CARDINALITE de relation :
  - le nombre de ses n-uplets (ou lignes). Rien à voir donc avec la cardinalité du modèle entité-association.
- DEGRE (ou encore ORDRE, ARITE) de relation :
  - le nombre d'attributs (ou colonnes). Rien à voir donc avec la dimension d'une association.

| CODE_CLIENT | NOM    | AD_RUE            | AD_VILLE  |
|-------------|--------|-------------------|-----------|
| AXV508      | DUPONT | 15, rue du Canada | CAEN      |
| BWY405      | DURAND | 6, rue Méchin     | CHERBOURG |
| KAC567      | DUVAL  | 25 rue Blanche    | ALENCON   |

ATTRIBUT

N-UPLET

Valeurs d'attribut

# Définition en intention d'une relation

---

**CLIENT** (Code client, Nom, Ad\_rue, Ad\_ville)

*Nom de relation*      *Clé de relation*      *Attributs*



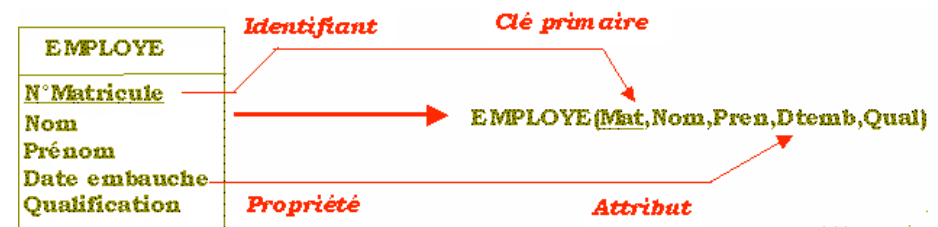
# OPERATEURS de manipulation des données :

---

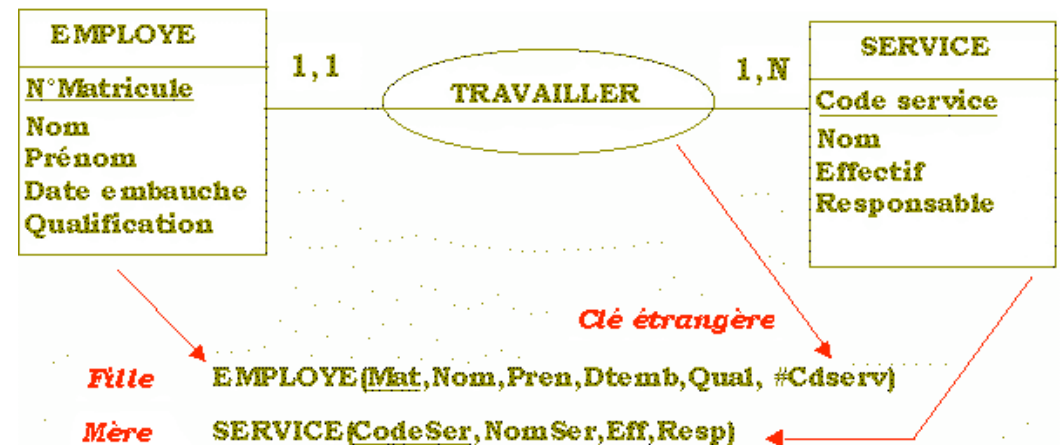
- L'UNION de 2 relations (définies sur les mêmes domaines fournit une nouvelle relation qui contient tous les n-uplets des 2 relations initiales.
- L'INTERSECTION de 2 relations (ayant mêmes domaines) fournit une nouvelle relation qui contient les n-uplets communs aux 2 relations initiales.
- La DIFFERENCE de 2 relations (ayant mêmes domaines) fournit une nouvelle relation qui contient les n-uplets de la 1ère qui ne se trouvent pas dans la 2ème.
- La PROJECTION d'une relation (opérateur unaire) fournit une nouvelle relation définie pour les seuls attributs demandés.
- La SELECTION d'une relation (opérateur unaire) fournit une nouvelle relation restreinte aux n-uplets qui respectent une proposition logique formulée à l'aide d'opérateurs de comparaison et /ou d'opérateurs logiques.
- La JOINTURE de 2 relations (ayant un attribut commun) fournit une nouvelle relation concaténant 2 à 2 les n-uplets des 2 relations initiales ayant même valeur pour l'attribut commun.
- 
- Chaque opérateur retourne un objet qui est lui-même une relation. Par conséquent, on peut ré appliquer l'un quelconque des opérateurs au résultat fourni par le précédent.

# Règles de passage modèle Entité-Association vers un schéma Relationnel

- RG°1 : TOUTE ENTITE DEVIENT UNE RELATION dans laquelle :
  - les attributs traduisent les propriétés de l'entité
  - la clé primaire traduit l'identifiant de l'entité

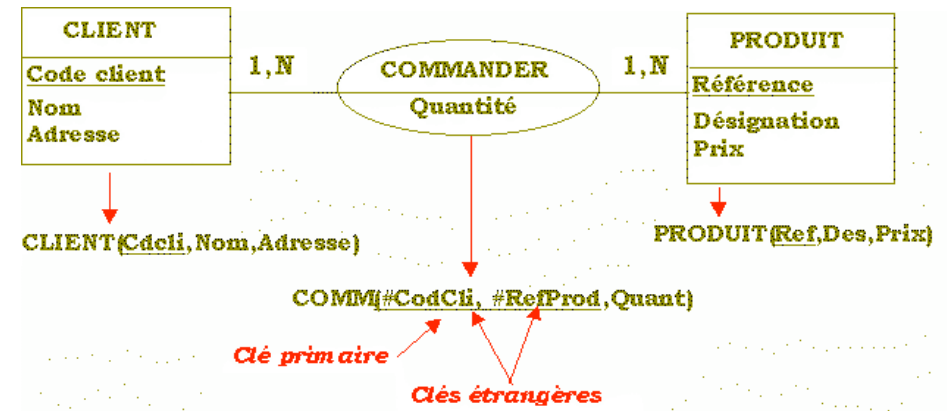


- RG °2 : UNE ASSOCIATION DE DIMENSION 2 AVEC CARDINALITE 1,1 SE REECRIT EN :
  - portant dans la relation fille la clé primaire de la relation mère.
  - L'attribut ainsi ajouté s'appelle clé étrangère. Par convention, nous la symboliserons au moyen de #.



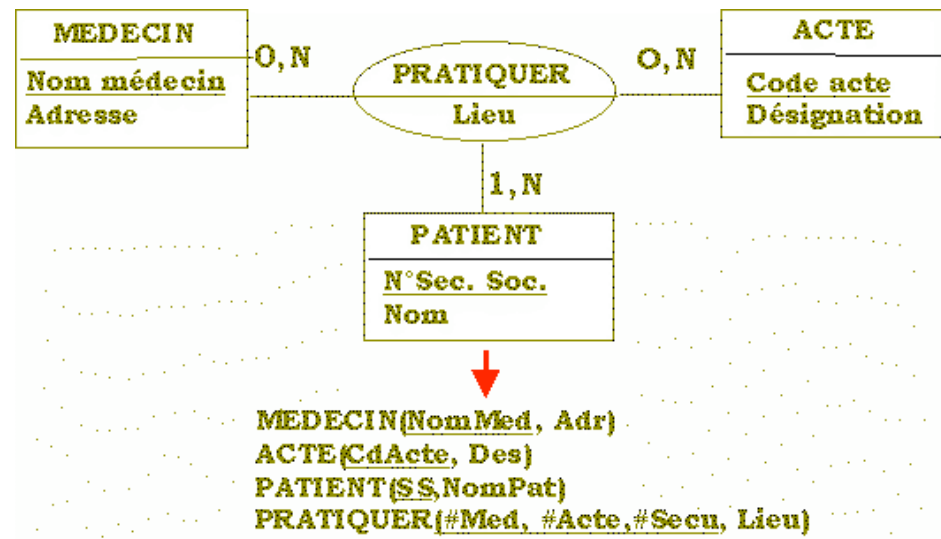
## Règles de passage modèle Entité-Association vers un schéma Relationnel

- RG°3 : UNE ASSOCIATION DE DIMENSION 2 AVEC CARDINALITES PLUSIEURS A PLUSIEURS SE REECRIT EN :
  - créant une relation particulière qui contient comme attributs les identifiants des 2 entités associées
  - ces attributs constituent à eux 2 la clé primaire de la relation
  - ils sont individuellement clés étrangères ajoutant la ou les éventuelles propriétés de l'association à cette relation.
- (Dans le cas de cardinalités minimales à 0, la règle s'applique de la même façon).



## Règles de passage modèle Entité-Association vers un schéma Relationnel

- RG°4 :
- UNE ASSOCIATION DE DIMENSION SUPERIEURE A 2 SE REECRIT SELON LA REGLE 3 :



# Conclusion

---

- Les règles de passage d'un modèle Entité-Association en schéma logique d'une Base de Données Relationnelles sont totalement formalisables
- *(pourvu que le modèle E-A soit au moins en 3ème Forme Normale)*
  - Par conséquent, on peut les automatiser au moyen d'un programme : c'est ce que font tous les Ateliers de Génie Logiciel qui vous aident à dessiner votre modèle E-A et vous génèrent ensuite tous seuls le schéma de la BD (on appelle souvent cela "génération de code", une appellation que l'on pourrait discuter)
- réf: Bernard Morand

# Sommaire

---

- Niveau logique
  - Fichiers
  - Bases de Données Relationnelles
  - Convention de traduction vers le niveau logique a travers un AGL
- Formes Normales

# Normalisation d'un schéma relationnel

---

- Si l'on n'y prend garde, une mauvaise répartition des données dans les relations peut occasionner de graves problèmes lors de l'évolution de la base.
- La normalisation des relations permet d'éviter ces problèmes. L'idée essentielle de la normalisation est : autant que possible un seul fait dans un seul lieu c'est-à-dire une seule notion sémantique par relation. Pour obtenir cela, le processus de normalisation conduit le plus souvent à éclater des relations en plusieurs relations. Il existe plusieurs degrés dans les formes normales.
- La notion de forme normale repose sur les notions
- de clé et de dépendance fonctionnelle.

# Normalisation d'un schéma relationnel

---

- Dépendances fonctionnelles
- Il y a une dépendance fonctionnelle entre deux attributs X et Y d'une même relation R (on dit aussi que Y dépend de X) si toute valeur pour l'attribut X détermine une valeur bien définie pour l'attribut Y. On note  $X \twoheadrightarrow Y$
- Les attributs X et Y peuvent être composés de plusieurs rubriques.
- Plus formellement, soient X et Y deux ensembles de rubriques de la relation R.
- $X \twoheadrightarrow Y$  (lire X détermine Y) si et seulement si  
E1 et E2 sont deux n-uplets de la relation R qui ont les mêmes valeurs pour les rubriques de X, alors ils ont nécessairement les mêmes valeurs pour les rubriques de Y.

# Normalisation d'un schéma relationnel

---

- Dépendances fonctionnelles
- $X \twoheadrightarrow Y$  (lire  $x$  détermine  $y$ ) si et seulement si  
E1 et E2 sont deux  $n$ -uplets de la relation  $R$  qui ont les mêmes valeurs pour les rubriques de  $X$ , alors ils ont nécessairement les mêmes valeurs pour les rubriques de  $Y$ .
  - (a) Il y a dépendance fonctionnelle d'une clé primaire vers tous les autres attributs d'une relation.
  - (b) Dans la relation (Nom, Prénoms, Ville, Code postal, Département), on a la dépendance fonctionnelle Code postal  $\twoheadrightarrow$  Département ;  
mais pas Ville  $\twoheadrightarrow$  Département, (car 2 villes peuvent avoir le même nom)
  - (c) Dans une classe d'étudiants sans homonymes (pour le nom et le prénom), dans la relation (Nom, Prénom, Moyenne, Âge, Enseignant), on a les dépendances fonctionnelles  
(Nom, Prénom)  $\twoheadrightarrow$  Moyenne  
(Nom, Prénom)  $\twoheadrightarrow$  Age

# Dépendances fonctionnelles, exemple

---

| affectations | PILOTE | VOL | DATE    | HEURE |
|--------------|--------|-----|---------|-------|
|              | Alpha  | 890 | 10.mars | 18:40 |
|              | Delta  | 836 | 10.mars | 18:35 |
|              | Alpha  | 836 | 12.mars | 18:35 |
|              | Bravo  | 758 | 10.mars | 20:40 |
|              | Bravo  | 890 | 11.mars | 18:40 |
|              | Alpha  | 890 | 13.mars | 18:40 |
|              | Bravo  | 830 | 10.mars | 07:45 |

- contraintes:
  - un vol part toujours à la même heure
    - le vol est fonction de l'heure
  - un pilote, à une date et une heure données, assure un seul vol
- dépendances fonctionnelles:
  - $VOL \rightarrow HEURE$ 
    - il existe une dépendance fonctionnelle de VOL vers HEURE dans Affectations
  - $\{PILOTE, DATE, HEURE\} \rightarrow VOL$
- dépendance stricte (totale):
  - cesse de l'être si on lui enlève un attribut déterminant

# Autre exemple de dépendance fonctionnelle

---

| commande | CLIENT | NUMERO | PRODUIT | DATE | QUANTITE | PRIX |
|----------|--------|--------|---------|------|----------|------|
|          |        |        |         |      |          |      |
|          |        |        |         |      |          |      |

*Si  $\langle c \ n \ p \ d \ q \ pu \rangle \in \text{commande}$ ,  
alors le client (c) a passé à la date (d) la commande (n) pour une quantité (q) du produit (p) au prix unitaire (pu)*

- Contraintes:
  - la numérotation des commandes est faite par compostage (sans remise à zéro)
  - on n'admet qu'une seule commande par client et par jour
  - le prix unitaire d'un produit ne dépend que de celui-ci
  - une même commande peut concerner plusieurs produits
    - mais alors ils sont différents
  - une commande est passée par un seul client

## Exemple e) de dépendance suite

| commande | CLIENT | NUMERO | PRODUIT | DATE | QUANTITE | PRIX |
|----------|--------|--------|---------|------|----------|------|
|          |        |        |         |      |          |      |
|          |        |        |         |      |          |      |

*Si  $\langle c \ n \ d \ q \ pu \rangle \in \text{commande}$ ,  
alors le client (c) a passé à la date (d) la commande (n) pour une quantité (q) du produit (p) au prix unitaire (pu)*

- Contraintes:
  - a) la numérotation des commandes est faite par compostage (sans remise à zéro)
  - b) on n'admet qu'une seule commande par client et par jour
  - c) le prix unitaire d'un produit ne dépend que de celui-ci
  - d) une même commande peut concerner plusieurs produits (mais alors ils sont différents)
  - e) une commande est passée par un seul client
- Dépendances:
  - **NUMERO  $\rightarrow$  CLIENT (e)**
  - **NUMERO  $\rightarrow$  DATE (a)**
  - **CLIENT, DATE  $\rightarrow$  NUMERO (b)**
  - **NUMERO, PRODUIT  $\rightarrow$  QUANTITE (d)**
  - **PRODUIT  $\rightarrow$  PRIX (c)**

# Propriétés des dépendances

---

- Réflexivité
  - $X \rightarrow X$
- Augmentation
  - si  $X \rightarrow Y$  alors  $X, Z \rightarrow Y$
- Transitivité
  - si  $X \rightarrow Y$  et  $Y \rightarrow Z$  alors  $X \rightarrow Z$
- Additivité
  - si  $X \rightarrow Y$  et  $X \rightarrow Z$  alors  $X \rightarrow Y, Z$
- Projection
  - si  $X \rightarrow Y, Z$  alors  $X \rightarrow Y$
- Pseudo-transitivité
  - si  $X \rightarrow Y$  et  $Y, Z \rightarrow W$  alors  $X, Z \rightarrow W$

# Schéma relationnel normalisé

---

- Un schéma relationnel est normalisé pour une certaine forme normale si les relations qui le composent sont toutes dans la forme normale spécifiée.
- Normaliser un schéma relationnel c'est le transformer en un schéma relationnel normalisé équivalent, c'est-à-dire, contenant les mêmes informations que le schéma de départ.
  - En général cela consiste à éclater les relations non normalisées en plusieurs relations normalisées qui redonnent les relations d'origine par jointure.
  - La décomposition doit être sans perte de données et autant que possible sans perte de dépendance
  - L'idée « un seul fait dans un seul lieu » se traduit souvent par la suppression des dépendances fonctionnelles intra relation autres que celles liées aux clés candidates.

# Schéma relationnel normalisé?

---

- Normalisation par décomposition
  - Cf. Supra
- Construction par synthèse

| commande | CLIENT | NUMERO | PRODUIT | ADRESSE | QUANTITE |
|----------|--------|--------|---------|---------|----------|
|          |        |        |         |         |          |

**CLIENT → ADRESSE**

**NUMERO → CLIENT**

**NUMERO, PRODUIT → QUANTITE**

relations:

**Clients (CLIENT, ADRESSE)**

**Entête (NUMERO, CLIENT)**

**LIGNE (NUMERO, PRODUIT, QUANTITE)**

clés étrangères:

CLIENT ...

NUMERO ...

# Pourquoi normaliser?

---

- Exemple La dépendance  $\text{Dept} \twoheadrightarrow \text{NomD}$   
dans la relation  $(\underline{\text{Matr}}, \text{NomE}, \text{Dept}, \text{NomD})$   
va occasionner des problèmes de perte de place sur les supports de la base dus à la redondance des données  
et des anomalies lors des modifications des données :
  - anomalies d'insertion : on ne peut ajouter de nouveau département (numéro et nom) si le département ne comporte pas d'employé,
  - anomalies de modification : si l'on change le nom d'un département, on doit le changer dans toutes les lignes des employés de ce département,
  - anomalies de suppression : si on supprime le dernier employé d'un département, on perd l'information numéro-nom du département.
- La normalisation de la relation  $(\underline{\text{Matr}}, \text{NomE}, \text{Dept}, \text{NomD})$   
par sa décomposition en deux relations  $(\underline{\text{Matr}}, \text{NomE})$   $(\underline{\text{Dept}}, \text{NomD})$   
aurait permis d'éviter les problèmes pendant la mise à jour des données.

# Formes normales

---

- Pour tous les exemples et contre-exemples cités ici, on supposera l'existence des dépendances fonctionnelles naturelles que l'on rencontrerait dans les cas concrets.
  - Par exemple, on suppose l'existence de la dépendance fonctionnelle  $\text{CodeL} \twoheadrightarrow \text{Titre}$
- 1NF 1ère forme normale: une relation est en première forme normale si elle ne comporte pas d'attribut multi valué.

La représentation des données sous forme de relation implique cette première forme normale.

  - Exemple de pseudo-relation qui n'est pas en 1NF:
  - $\text{LIVRE}(\text{CodeL}, \text{Titre}, \text{Auteurs})$  n'est pas en première forme normale car un livre pourrait avoir plusieurs auteurs. Solution:
  - $\text{LIVRE}(\text{CodeL}, \text{Titre}), \text{AUTEURS}(\text{CodeL}, \text{Auteur})$

# Formes normales

---

2NF 2ème forme normale : une relation est en deuxième forme normale si la relation est en première forme normale et si chaque attribut non-clé (c'est-à-dire qui n'appartient pas à une clé candidate) dépend pleinement des clés candidates.

- Une dépendance fonctionnelle d'un attribut par rapport à un ensemble d'attributs est pleine si l'attribut ne dépend pas d'un sous-ensemble strict de cet ensemble d'attributs. On dit aussi que la dépendance fonctionnelle est élémentaire.

(unicité, décomposition minimale ou dfe)

- Exemple de relation qui n'est pas en 2NF:  
EMP(Matr, CodeP, NomE, FonctionP, Dept) n'est pas en 2NF car le nom d'un employé ou son département ne dépend que de son matricule et pas du projet auquel il appartient.

# Formes normales

---

Exemple de relation qui n'est pas en 2NF:

EMP(Matr, CodeP, NomE, FonctionP, Dept)

n'est pas en 2NF car le nom d'un employé ou son département ne dépend que de son matricule et pas du projet auquel il appartient.

- Problèmes occasionnés lors de la mise à jour des données : si on modifie le nom d'un employé, on doit le modifier dans toutes les lignes qui correspondent aux divers projets
- on ne peut insérer un nouvel employé s'il ne participe pas à un projet
- on risque de supprimer les renseignements sur un employé s'il ne participe plus à aucun projet.
- On peut décomposer cette relation en deux relations qui seront en 2NF  
EMP(Matr, NomE, Dept)  
PARTICIPATION(Matr, CodeP, FonctionP)

# Formes normales

---

3NF 3ème forme normale : une relation est en troisième forme normale si tout attribut non-clé (n'appartenant pas à une clé candidate) ne peut dépendre que d'une clé candidate.

- Dans le cas où il n'y a qu'une seule clé candidate, cette définition est équivalente à la suivante :
  - la relation est en deuxième forme normale et il y a dépendance directe (non transitive, c'est-à-dire une dépendance qui pourrait être retrouvée par composition de deux dépendances fonctionnelle) des attributs qui n'appartiennent pas à la clé par rapport à clé primaire.
- Remarque 2
  - Les dépendances fonctionnelles vers une rubrique qui appartient à une clé sont autorisées. C'est ce qui fait la différence avec la forme normale de Boyce-Codd

# Formes normales

---

3NF 3ème forme normale : une relation est en troisième forme normale si tout attribut non-clé (n'appartenant pas à une clé candidate) ne peut dépendre que d'une clé candidate.

- Exemple **EMP(Matr, NomE, Dept, NomD)** où Dept est le numéro du Département n'est pas en 3NF car le nom du département dépend du numéro du département.

Comme pour la 2NF, on rencontrera des anomalies lors des mises à jour de relations qui ne sont pas en 3NF.

On sera par exemple obligé de dupliquer les renseignements concernant les départements.

On peut décomposer cette relation en deux relations qui seront en 3NF

**EMP(Matr, NomE, Dept)** et **DEPT(Dept, NomD)**

# Formes normales

---

- BCNF Forme normale de Boyce-Codd : une relation est en forme normale de Boyce-Codd si les seules sources de dépendances fonctionnelles sont les clés candidates.
  - Ceci implique que la relation est en troisième forme normale mais une relation peut être en troisième forme normale sans être dans la forme normale de Boyce-Codd. Cela ne peut arriver que si deux clés candidates se chevauchent (ont en commun une ou plusieurs rubriques) et qu'une partie d'une des clés candidates dépend d'un attribut qui appartient à l'autre clé candidate.
- L'avantage de travailler avec des relations en forme normale de Boyce-Codd est que les seules dépendances fonctionnelles intra relation à respecter lors des ajouts ou modifications de données de la base sont celles qui sont liées à l'unicité des clés d'une relation.

# Formes normales

---

- BCNF Forme normale de Boyce-Codd : une relation est en forme normale de Boyce-Codd si les seules sources de dépendances fonctionnelles sont les clés candidates.
  - Exemples 5
    - (a) Si on suppose que le nom d'un employé peut être une clé candidate, la relation EMP\_PROJET(Matr, CodeP, NomE, FonctionP) est en 3NF. En effet les 2 clés candidates sont (Matr, CodeP) et (NomE, CodeP)
    - et le seul attribut qui n'appartient pas à une clé candidate est FonctionP qui ne dépend pas d'un sous-ensemble strict d'une des clés candidates.
    - Cette relation n'est pas en BCNF car Matr --> NomE est une dépendance fonctionnelle dont la source n'est pas une clé candidate. Elle va occasionner des redondances et des problèmes de mises à jour. Elle peut être décomposées dans les 2 relations suivantes qui sont en BCNF:  
EMP(Matr, NomE)   PROJET(Matr, CodeP, FonctionP)
    -

# Formes normales

---

(b) Un cas plus intéressant correspond au schéma suivant d'une relation :

(clé1, clé2, rubrique3)

avec les dépendances fonctionnelles

(clé1, clé2)  $\rightarrow$  rubrique3

rubrique3  $\rightarrow$  clé2

est en troisième forme normale mais pas en forme normale de Boyce-Codd (voir remarque ci-dessus sur la troisième forme normale). On verra un exemple de cette situation dans la section décomposition avec perte de dépendance.

# Autre formulation de la 1ère forme normale

| vols | VOL   | AEROPORT  | JOUR     | PILOTE |        |
|------|-------|-----------|----------|--------|--------|
|      |       |           |          | PILOTE | QUALIF |
|      | SN890 | Genève    | lundi    | Alpha  | 1PL    |
|      |       |           | jeudi    | Bravo  | 2PL    |
|      | SN891 | Bruxelles | lundi    | Alpha  | 1PL    |
|      |       |           | jeudi    | Delta  | 1PL    |
|      | SR836 | Genève    | mercredi | Bravo  | 2PL    |

- Un schéma de relation est en 1ère forme normale si
  - le domaine de ses attributs est simple (atomique)
    - non décomposable, non répétitif

| volsN | VOL   | AEROPORT  | JOUR     | PILOTE | QUALIF |
|-------|-------|-----------|----------|--------|--------|
|       | SN890 | Genève    | lundi    | Alpha  | 1PL    |
|       | SN890 | Genève    | jeudi    | Bravo  | 2PL    |
|       | SN891 | Bruxelles | lundi    | Alpha  | 1PL    |
|       | SN891 | Bruxelles | jeudi    | Delta  | 1PL    |
|       | SR836 | Genève    | mercredi | Bravo  | 2PL    |

# Autre formulation de la 2ème forme normale

| <b>volsN</b> | <b><u>VOL</u></b> | <b><u>JOUR</u></b> | AEROPORT  | PILOTE | QUALIF |
|--------------|-------------------|--------------------|-----------|--------|--------|
|              | SN890             | lundi              | Genève    | Alpha  | 1PL    |
|              | SN890             | jeudi              | Genève    | Bravo  | 2PL    |
|              | SN891             | lundi              | Bruxelles | Alpha  | 1PL    |
|              | SN891             | jeudi              | Bruxelles | Delta  | 1PL    |
|              | SR836             | mercredi           | Genève    | Bravo  | 2PL    |

**VOL → AEROPORT**

- Un schéma de relation est en 2ème forme normale si
  - elle est en 1ère forme et si
  - tout attribut non identifiant dépend strictement de chaque identifiant

| <b>volsA</b> | <b><u>VOL</u></b> | AEROPORT  |
|--------------|-------------------|-----------|
|              | SN890             | Genève    |
|              | SN891             | Bruxelles |
|              | SR836             | Genève    |

| <b>volsP</b> | <b><u>VOL</u></b> | <b><u>JOUR</u></b> | PILOTE | QUALIF |
|--------------|-------------------|--------------------|--------|--------|
|              | SN890             | lundi              | Alpha  | 1PL    |
|              | SN890             | jeudi              | Bravo  | 2PL    |
|              | SN891             | lundi              | Alpha  | 1PL    |
|              | SN891             | jeudi              | Delta  | 1PL    |
|              | SR836             | mercredi           | Bravo  | 2PL    |

# Autre formulation de la 3ème forme normale

| volsP | <u>VOL</u> | <u>JOUR</u> | PILOTE | QUALIF |
|-------|------------|-------------|--------|--------|
|       | SN890      | lundi       | Alpha  | 1PL    |
|       | SN890      | jeudi       | Bravo  | 2PL    |
|       | SN891      | lundi       | Alpha  | 1PL    |
|       | SN891      | jeudi       | Delta  | 1PL    |
|       | SR836      | mercredi    | Bravo  | 2PL    |

**PILOTE → QUALIF**

- Un schéma de relation est en 3ème forme normale si
  - elle est en 2ème forme et si
  - aucun attribut non identifiant ne dépend transitivement d'un identifiant

| pilotes | <u>PILOTE</u> | QUALIF |
|---------|---------------|--------|
|         | Alpha         | 1PL    |
|         | Bravo         | 2PL    |
|         | Delta         | 1PL    |

| nouvAffect | VOL   | JOUR     | PILOTE |
|------------|-------|----------|--------|
|            | SN890 | lundi    | Alpha  |
|            | SN890 | jeudi    | Bravo  |
|            | SN891 | lundi    | Alpha  |
|            | SN891 | jeudi    | Delta  |
|            | SR836 | mercredi | Bravo  |



# Autre formulation de la Forme de *Boyce-Codd*

| nouvAffect | VOL   | JOUR     | PILOTE |
|------------|-------|----------|--------|
|            | SN890 | lundi    | India  |
|            | SN890 | jeudi    | India  |
|            | SN891 | jeudi    | Sierra |
|            | SR836 | mercredi | Tango  |

**PILOTE → VOL**

- Un schéma de relation est en 3ème forme normale de *Boyce-Codd* si
  - elle est en 1ère forme et si
  - tout déterminant strict est un identifiant

| affectPV | PILOTE | VOL   |
|----------|--------|-------|
|          | India  | SN890 |
|          | Sierra | SN891 |
|          | Tango  | SR836 |
|          |        |       |

| affectJP | JOUR     | PILOTE |
|----------|----------|--------|
|          | lundi    | India  |
|          | jeudi    | India  |
|          | jeudi    | Sierra |
|          | mercredi | Tango  |

mais **VOL, JOUR → PILOTE** dans **affectPV JOIN affectJP**

# Sommaire

---

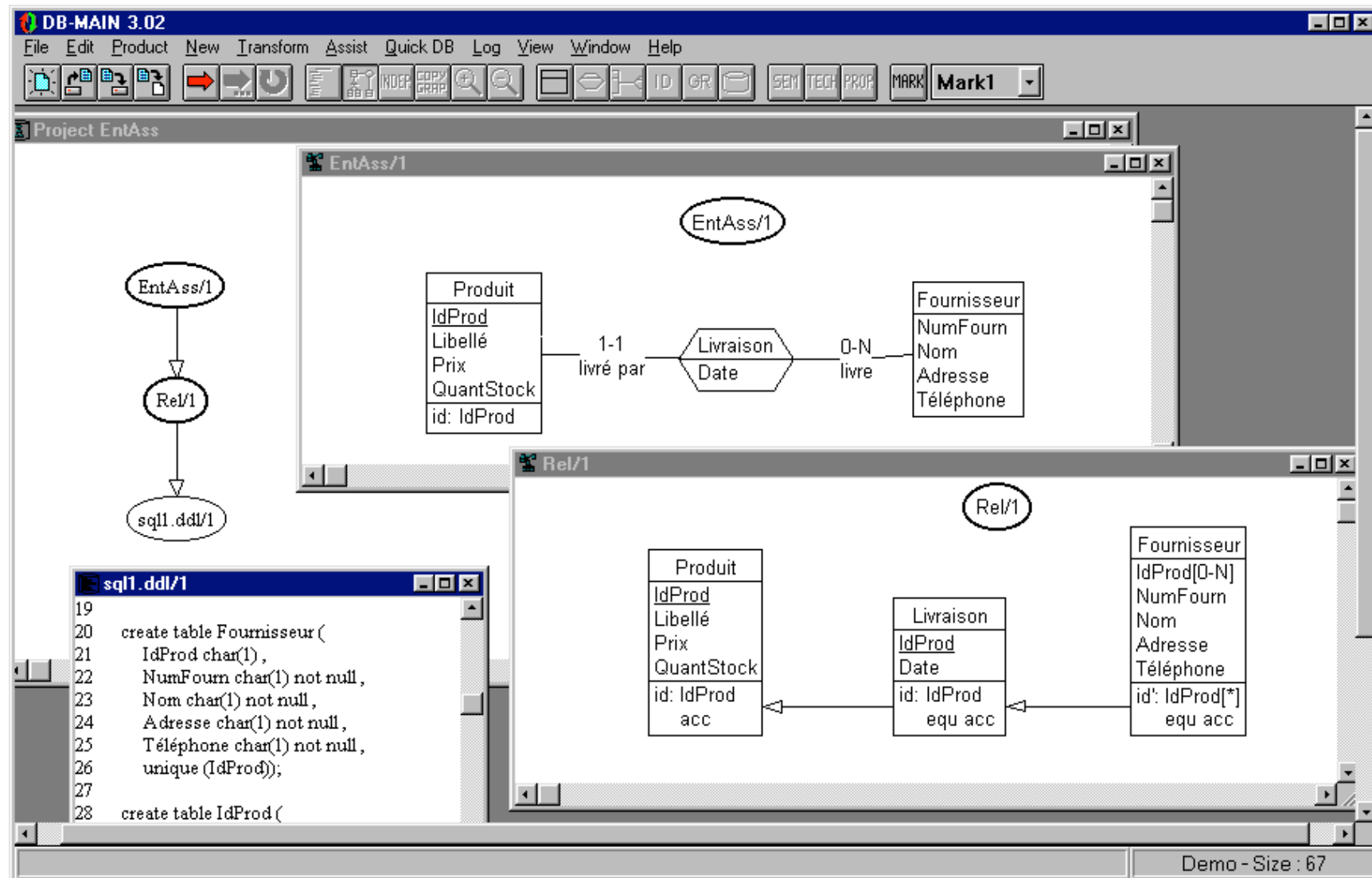
- Niveau logique
  - Fichiers
  - Bases de Données Relationnelles
  - Convention de traduction vers le niveau logique a travers un AGL
- Formes Normales

# Passage MCD MLD

---

- Faire TP sur les  
Convention de traduction vers le niveau logique à travers un AGL: les auditeurs doivent réaliser le passage du MCD au MLD en considérant successivement:
  - les associations binaires  $1,1 - (0 \text{ ou } 1), n$
  - les associations binaires  $(0 \text{ ou } 1), n - (0 \text{ ou } 1), n$  non porteuse de données
  - les associations binaires  $1, n - 1, n$  porteuse de données
  - les associations binaires  $0, n - 1, n$  porteuse de données
  - les associations binaires  $1, 1 - 0, 1$  non porteuse de données
  - les associations ternaires  $0, n - 0, n - 0, N$  non porteuse de données
  - les associations ternaires  $0, n - 0, n - 0, N$  porteuse de données
  - les associations binaire  $1, 1 - 1, n$  avec un identifiant relatif (référence)
- Vous réalisez une seule association de chacun des types ci-dessus, vous demandez à l'AGL de vous en donner le MLD et vous observez la représentation obtenue.
- Pourquoi ce TP? Le MLD est ce que vous obtiendrez en retro conception. Il faut savoir le lire. Aussi, les leçons 9 et 11 de DBMain seront plus faciles la semaine prochaine.

# Passage MCD MLD



# Formes normales : Pertes de données et de dépendances

---

- Les pertes de données ou de dépendances peuvent survenir lors des décompositions des relations

# Formes normales : Pertes de données et de dépendances

---

- Perte de données

- Une décomposition en plusieurs relations est dite sans perte de données si la jointure naturelle (voir 4.4.1) des relations obtenues dans la décomposition redonne la relation de départ.
- Voici un exemple de décomposition avec perte de données : Soit la relation EMP\_DEPT(Matr, Dept, NomE, Salaire, NomD)

avec les dépendances fonctionnelle

Matr --> NomE et Dept ---> NomD

- Cette relation ne peut être décomposé en EMP(Mat, NomE, Salaire) DEPT(Dept, NomD, Salaire) car la jointure naturelle de ces deux relations sur l'attribut Salaire donnera plus de n-uplets que la relation de départ si plusieurs employés ont le même salaire (la perte de données est en fait un surplus de données parasites). Ceci est dû au fait que la rubrique commune (Salaire) n'est pas source d'une dépendance fonctionnelle vers les autres rubriques d'une des deux relations de la décomposition.

# Formes normales : Pertes de données et de dépendances

---

- Perte de données
  - Une décomposition en plusieurs relations est dite sans perte de données si la jointure naturelle (voir 4.4.1) des relations obtenues dans la décomposition redonne la relation de départ.
- Soit une relation  $R(A, B, C)$  où  $A, B, C$  sont des ensembles d'attributs disjoints avec la dépendance fonctionnelle  $B \twoheadrightarrow C$ . On peut démontrer que la décomposition de  $R$  en les relations  $R_1(A \cup B)$  et  $R_2(B \cup C)$  est sans perte de données.
  - Comparez avec la décomposition page précédente avec perte de dépendance avec la suivante :  $EMP(\underline{Matr}, NomE, Salaire, Dept)$      $DEPT(\underline{Dept}, NomD)$   
Ici l'attribut commun est la source de la dépendance fonctionnelle  $Dept \twoheadrightarrow NomD$  et on n'a donc pas de perte de données.

# Perte de dépendances

---

- Une décomposition en plusieurs relations est dite sans perte de dépendances si on peut retrouver logiquement (par exemple par transitivité) les dépendances de départ à partir de la projection de l'ensemble des dépendances de départ sur les relations de la décomposition
- (on obtient l'ensemble projection sur une des relation  $R_1$  de la décomposition en ne gardant des dépendances de départ que celles dont les attributs sont tous des attributs de la relation  $R_1$ ).
  - Si on décompose une relation en plusieurs relations avec perte de dépendance, on ne peut travailler indépendamment dans chacune des relations de la décomposition sans s'occuper des autres relations. Lors des mises à jour, on ne peut en effet s'assurer que les dépendances perdues sont vérifiées qu'en consultant les autres relations, ce qui peut occasionner des traitements coûteux.

# Perte de dépendances

---

- Lorsque l'on a le choix entre plusieurs décompositions il faut évidemment préférer celles qui préservent les dépendances. Il est cependant des cas où il faut choisir entre une perte de dépendance ou une perte de normalisation comme dans l'exemple suivant :

# Fermeture et couverture

---

Soient les dépendances fonctionnelles d'une relation

- Fermeture transitive
  - ensemble de dépendances, contenant l'ensemble initial, tel que les règles de (pseudo-) transitivité ne peuvent plus être appliquées pour en dériver de nouvelles
  - ensemble de tous les faits exprimés (ou non) dans la relation
- Couverture minimale
  - ensemble minimum de dépendances, équivalent à la fermeture transitive
    - peut ne pas être unique
  - ensemble minimum des faits élémentaires d'une relation

# Exemple de couverture

---

- **horaire (PROF, MATIERE, HEURE, ANNEE, LOCAL)**
  - *si  $(p, m, h, a, l) \in \text{horaire}$ ,*
  - *alors un professeur  $(p)$  enseigne la matière  $(m)$  à l'heure  $(h)$  aux étudiants de l'année  $(a)$  dans le local  $(l)$*
- Contraintes :
  - un professeur n'enseigne qu'une matière
  - l'horaire se répète de semaine en semaine
  - HEURE désigne l'une des 40 heures de la semaine
  - les cours se donnent par heures entières
  - deux années n'ont pas cours en même temps au même endroit

# Exemple de couverture suite

---

- Ensemble initial de d.f. :

- 1. PROF → MATIERE
- 2. HEURE ANNEE → PROF
- 3. HEURE, LOCAL → ANNEE
- 4. PROF, HEURE → LOCAL
- 5. HEURE, ANNEE → MATIERE
- 6. PROF, HEURE → ANNEE

- Par transitivité:

- 7. HEURE, ANNEE → LOCAL (2. et 4.)
- 8. HEURE, LOCAL → PROF (3. et 2.)
- 9. HEURE, LOCAL → MATIERE (3. et 5.)

# Exemple de couverture <sub>fin</sub>

---

- Dérivation d'une première couverture minimale :

|      |                     |   |                |                       |
|------|---------------------|---|----------------|-----------------------|
| * 1. | <b>PROF</b>         | → | <b>MATIERE</b> | (non déductible)      |
| — 2. | HEURE ANNEE         | → | PROF           | (déduite de 7 et 8)   |
| — 3. | HEURE, LOCAL        | → | ANNEE          | (déduite de 8 et 6)   |
| — 4. | PROF, HEURE         | → | LOCAL          | (déduite de 6 et 7)   |
| — 5. | HEURE, ANNEE        | → | MATIERE        | (déduite de 2 et 1)   |
| * 6. | <b>PROF, HEURE</b>  | → | <b>ANNEE</b>   | (protégée par 3 et 4) |
| * 7. | <b>HEURE, ANNEE</b> | → | <b>LOCAL</b>   | (protégée par 2 et 4) |
| * 8. | <b>HEURE, LOCAL</b> | → | <b>PROF</b>    | (protégée par 2 et 3) |
| — 9. | HEURE, LOCAL        | → | MATIERE        | (déduite de 8 et 1)   |

- *autre couverture:*

|                |           |
|----------------|-----------|
| — PROF         | → MATIERE |
| — HEURE ANNEE  | → PROF    |
| — HEURE, LOCAL | → ANNEE   |
| — PROF, HEURE  | → LOCAL   |

# Clé et dépendances

---

- Clé (ou identifiant):
  - attribut (une liste d 'attributs) forme un identifiant s 'il est un déterminant, non nécessairement strict, pour chaque attribut de la relation
- $R(\underline{A}, B, C, D) \quad \Leftrightarrow \quad \begin{array}{l} A, B \rightarrow A \\ A, B \rightarrow B \\ A, B \rightarrow C \\ A, B \rightarrow D \end{array}$

# Détection des clés

---

- Algorithme (idée de solution):
- (distribuer le texte des algorithmes)
- Faire la suite du TD1 sur les DF et la normalisation pour introduire la notion de couverture ir redondante.
- Faire le TD Bravo pour introduire les algorithmes de résolutions

- 
- Faire la suite du TD1 sur les DF et la normalisation pour introduire la notion de couverture ir redondante.
  - Faire le TD Bravo pour introduire les algorithmes de résolutions